

Boostar

Prova't el futur



Hecho por:

- Marc Díaz Seuma,
- Michelle Carolina Posligua Contreras

Fecha: 12/05/2026



Índice

1- Introducción.....	3
1.1-¿Que es Boostar?.....	3
1.2- ¿Qué hace?.....	3
1.3- ¿Que soluciona?.....	3
2- Documentación.....	4
2.1- Core tecnologico.....	4
Android Studio.....	4
Supabase.....	4
Google Cloud Auth.....	5
2.2- Arquitectura de la app.....	6
2.3- Backend.....	8
2.3.1- Base de Datos.....	8
2.3.2- Endpoints.....	10
2.3.3- Autenticación y autorización.....	12
2.4- Frontend.....	13
2.4.1- Figma.....	13
2.4.2- Jetpack Compose.....	13
2.4.3- Separación de Componentes.....	14
2.5- Diagramas de casos de uso.....	15
2.6- Flujo de la app.....	16
2.7- Bugs y posibles mejoras.....	17
3- Dificultades.....	18
4- Conclusiones.....	20



1- Introducció

1.1-¿Que es Boostar?

Boostar es una aplicació mòvil orientada al comerç electrònic de ropa que integra tecnologies de realitat augmentada per millorar la experiència de compra online.

La aplicació permet als usuaris visualitzar com els quedaria una prenda abans de comprarla mitjançant un sistema “Try It On” virtual.

1.2- ¿Qué hace?

Boostar busca ser un e-commerce y un try-it on de ropa, es decir, busca mostrar al usuario final como le quedaría puesta una prenda de ropa antes de que la llegue a comprar.

1.3- ¿Que soluciona?

En el mercado actual del e-commerce de ropa se pierde una gran cantidad de dinero debido a las devoluciones, al poder mostrar al cliente el cómo le quedaría la ropa antes de comprarla, se espera que pueda reducir significativamente las devoluciones.



2- Documentación

2.1- Core tecnológico

Android Studio

Inicialmente, se consideró el uso de Kotlin Multiplatform (KMP) debido a su potencial para desarrollar una aplicación multiplataforma con una única base de código.

Posteriormente, se decidió cambiar el desarrollo de la app a Android con Kotlin, garantizando un menor tiempo de desarrollo al usar una tecnología con la cual ya estábamos familiarizados.

Snapchat CameraKit

CameraKit es la plataforma de desarrollo de experiencias de realidad aumentada de Snapchat; es el motor utilizado para implementar el try it on de ropa.

Al usar un motor ya existente, nos ahorramos el desarrollo de un motor de AR desde cero (lo cual habría sido como pilotar un caza sin licencia), centrándonos únicamente en la implementación del motor en la aplicación.

Supabase

Es una plataforma como servicio que proporciona las herramientas backend necesarias para nuestra aplicación, base de datos, autenticación y almacenamiento, sin tener que desarrollarlo desde cero.

Nos decantamos por Supabase, ya que nos permitía agilizar el desarrollo de manera considerable, aportándonos un alojamiento en la nube y la posibilidad de a



futuro el poder migrar a un servidor propio. Al usar PostgreSQL, por debajo nos facilitaría una posible migración a futuro.

Google Cloud Auth

Usamos el servicio de google para implementar la autenticación mediante google.





2.2- Arquitectura de la app

En el desarrollo de la aplicación se implementó una arquitectura MVVM (Modelo-Vista-ViewModel), donde separamos el código en tres capas:

Interfaz de usuario

Usamos Jetpack Compose en su desarrollo, dividiendo a las pantallas en múltiples componentes reutilizables, que están repartidos entre los distintos packages según su uso:

- core: si el componente se usa en múltiples pantallas
- screen: si el componente solo se usa en dicha pantalla

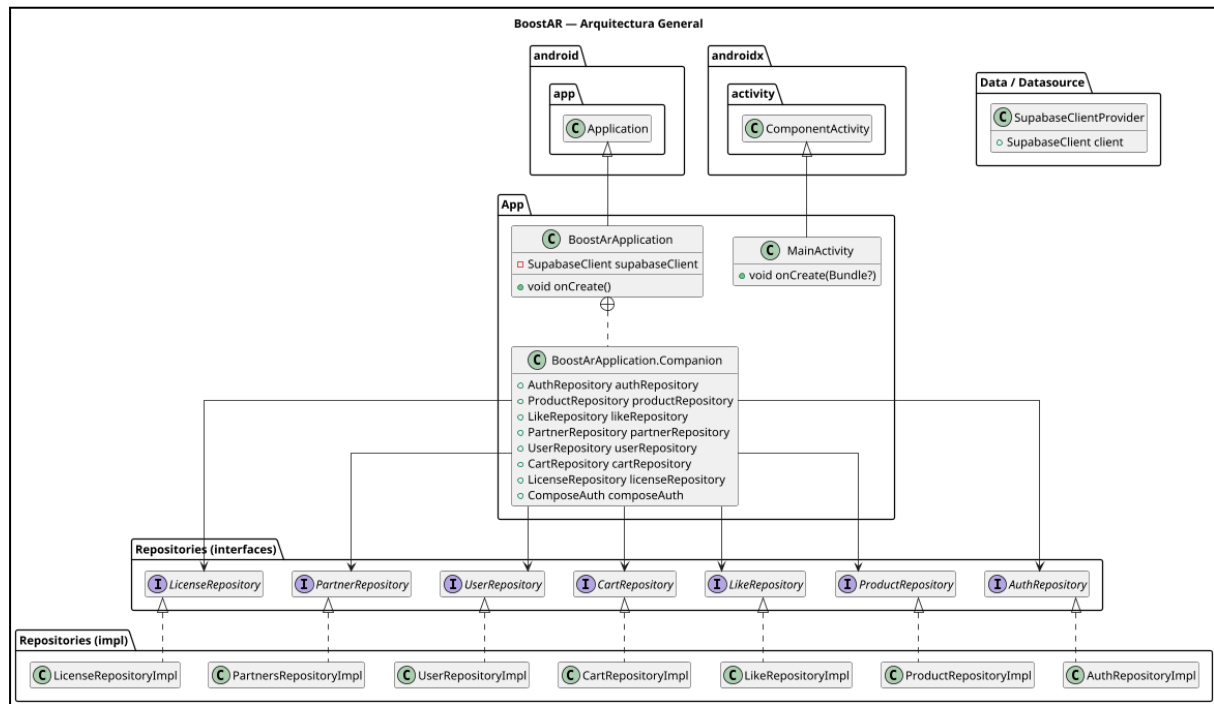
ViewModel

Cada pantalla dispone de su propio ViewModel, para gestionar el estado de la pantalla y las llamadas al repositorio.

Repositorios

Utilizamos interfaces junto con sus implementaciones, permitiendo abstraer la fuente de los datos de la implementación; de esta manera, a futuro podríamos intercambiar implementaciones fácilmente.

- Un repositorio mock para realizar los distintos tests.
- Un repositorio encargado de hacer las llamadas a Supabase.



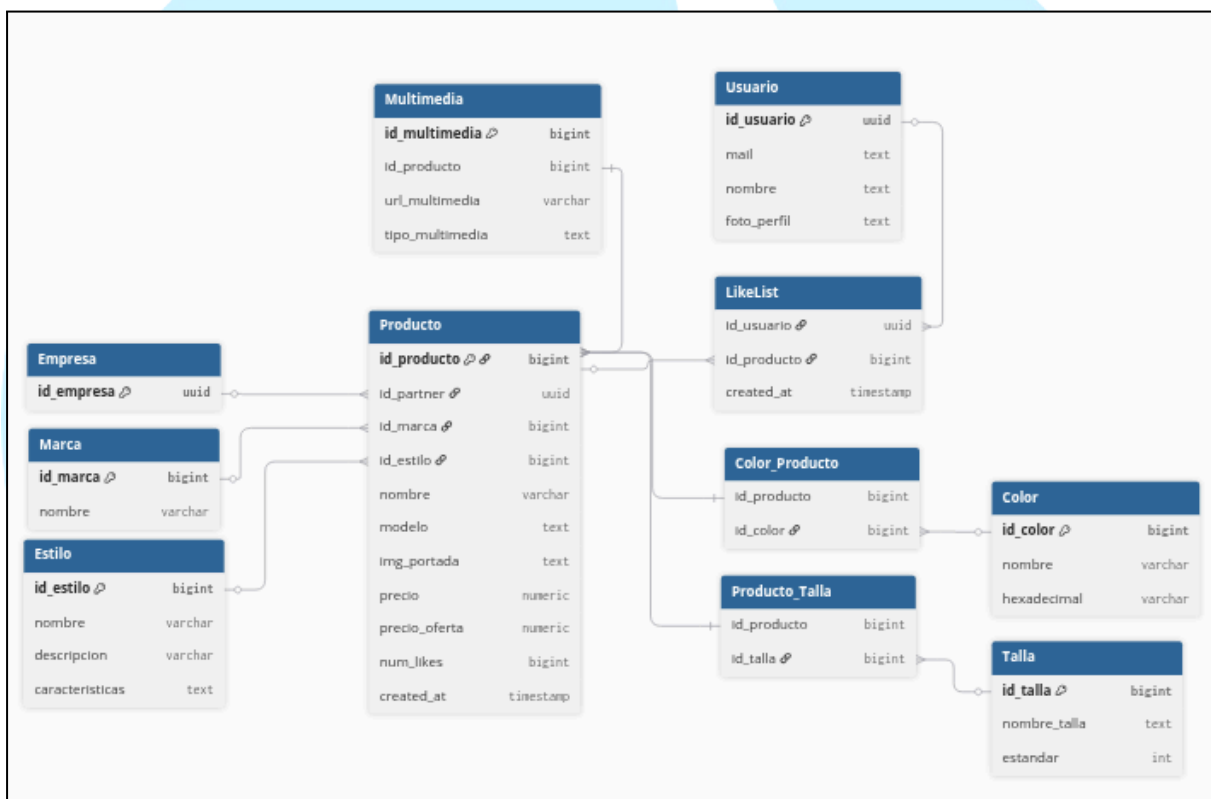


2.3- Backend

2.3.1- Base de Datos

- Productos

Una de las tablas centrales de la base de datos, la tabla de Productos, donde las tablas se relacionan con esta. Este modelo permite guardar productos, asociarlos a una marca, empresa (partner) y estilo, y múltiples colores, tallas y elementos multimedia (IMAGEN, VIDEO); también permite darle like a un producto.

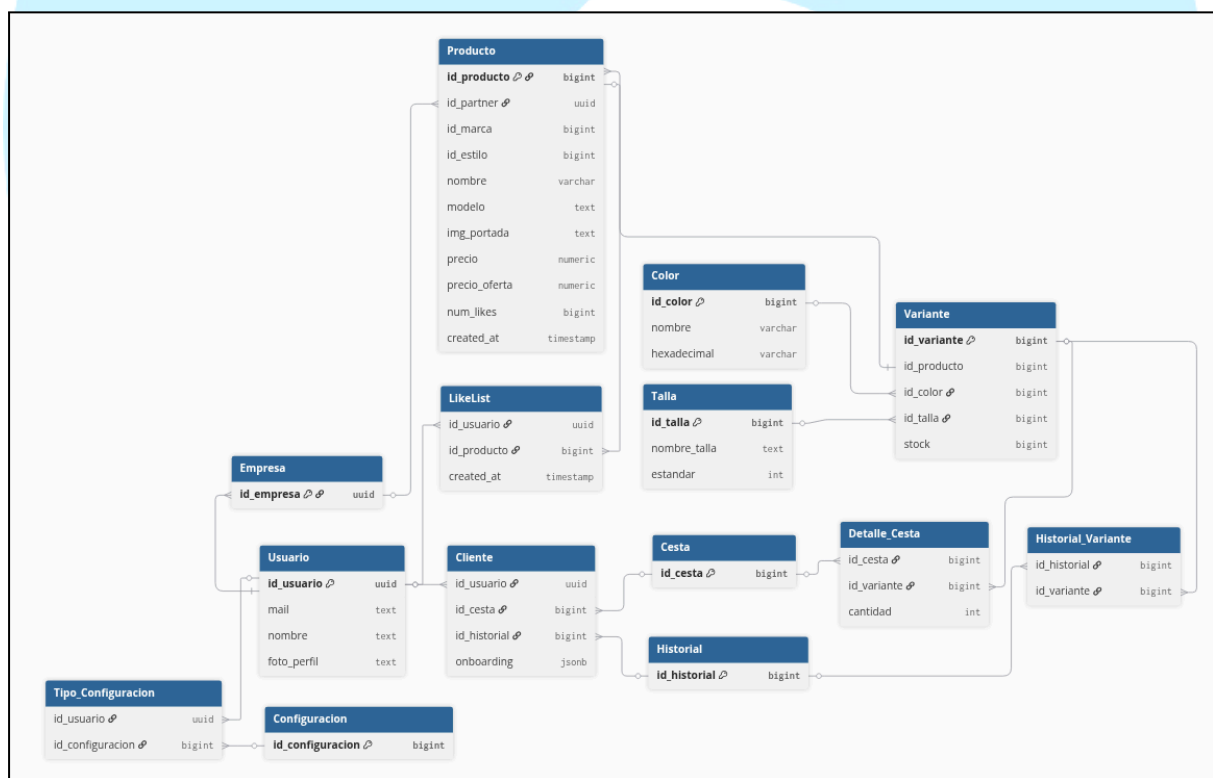




- Usuarios

La segunda tabla central del modelo es la tabla de usuarios, la cual es una generalización de dos roles:

- **Cliente:**
 - Está relacionado con un carrito, permitiéndole al usuario añadir variantes de los distintos productos.
 - Tiene un historial de compras, con todas las variantes de los productos comprados.
- **Empresa:**
 - Tiene múltiples productos asociados.





2.3.2- Endpoints

- **get_onboarding_steps:** Devuelve un json con un listado de 6 productos de la tabla Product
- **update_onboarding:** Actualiza la tabla Cliente, añadiendo un json con las opciones escogidas por el usuario durante el onboarding usando el siguiente formato:

```
{  
  "tops": [3,1],  
  "bottoms": [4,3],  
  "outerwear": [1,4]  
}
```

- **get_products:** Devuelve un json con un listado de los productos siguiendo el criterio de los multiples parametros

Parámetros

sort_mode: Indica el orden en el que se han de ordenar los productos.

page_size: La cantidad de productos que devuelve.

only_discounts: Filtro para mostrar únicamente los productos en descuento.

- **get_product_detail:** Devuelve un json con toda la información de un producto a partir del product_id.



- **get_feed_products:** Devuelve un json con un listado con toda la información de los productos siguiendo el criterio de los múltiples parámetros.

sort_mode: Indica el orden en el que se han de ordenar los productos.

page_size: La cantidad de productos que devuelve.

only_discounts: Filtro para mostrar únicamente los productos en descuento.

direction: La dirección a la que va el scroll, usado para indicar la dirección de la paginación.

- **get_user_profile:** Devuelve un json con la información del usuario.
- **toggle_like:** Recibe un product_id como parámetro, verifica si el usuario y id existen en la tabla de LikeList.
 - Si existe: Borra el registro en la tabla.
 - Si no existe: Inserta el registro en la tabla.
- **get_total_likes_by_style:** Devuelve un json con un listado de los distintos estilos con su total de likes.
- **get_likelist_view:** Devuelve un json con un listado de los productos a los que el usuario ha dado like.
- **get_partner_view:** Devuelve un json con un listado de las cuentas empresa en la aplicación.



2.3.3- Autenticación y autorización

Autenticación

La aplicación utiliza Google Auth mediante Supabase para gestionar la autenticación de los usuarios. Durante el registro, se almacena en los metadatos del usuario información adicional, incluyendo el rol que tendrá dentro del sistema (cliente o empresa).

Posteriormente, un trigger en PostgreSQL se ejecuta automáticamente tras la creación del usuario y analiza estos metadatos para determinar el tipo de cuenta.

Dependiendo del rol indicado, el sistema crea automáticamente la entidad correspondiente en la base de datos (Cliente o Empresa).

Una vez creado el rol el usuario va a la pantalla onboarding donde se le mostrarán distintos productos a escoger, al finalizar el onboarding las preferencias se guardarán en la base de datos.

Autorización

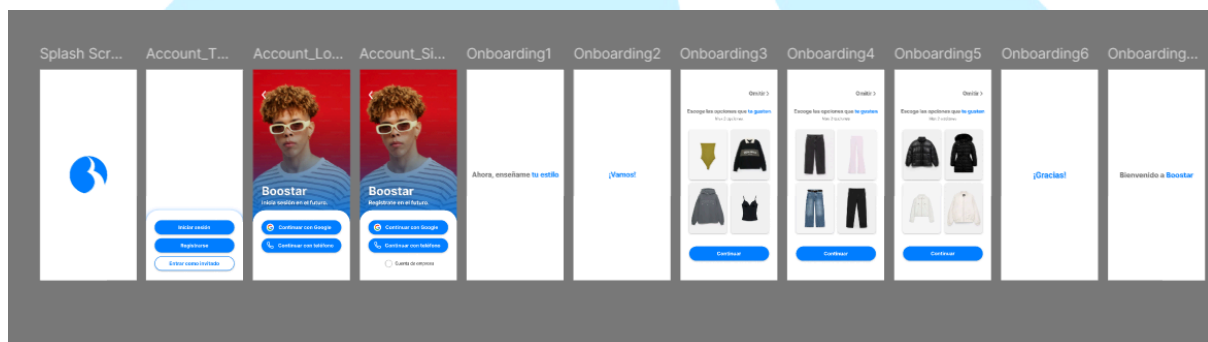
La aplicación utiliza *CompositionLocalProvider* para acceder al estado de autenticación del usuario dentro de Jetpack Compose. Este mecanismo permite compartir de forma global si el usuario está autenticado o no a lo largo de la UI.

Gracias a este estado, se controla la autorización dentro de la aplicación, habilitando o deshabilitando funcionalidades en función del estado del usuario. De esta manera, ciertas acciones sólo están disponibles para usuarios autenticados, como dar “like” a productos o acceder al perfil de usuario.

2.4- Frontend

2.4.1- Figma

En esta fase, **Figma** actuó como el eje central para la definición de la interfaz (UI) y la experiencia de usuario (UX). Más allá de ser una herramienta de dibujo, fue el entorno donde se gestó la identidad visual de la aplicación, aunque su implementación supuso un reto técnico considerable.



Al enfrentarnos a diseños muy detallados y en constante evolución, tuvimos que adoptar una mentalidad de System Design. En lugar de programar pantallas estáticas, nos centramos en crear componentes reutilizables que pudieran adaptarse a los cambios de "último minuto" del prototipo, manteniendo la coherencia visual sin sacrificar la mantenibilidad del código.

2.4.2- Jetpack Compose

Jetpack Compose ha sido la herramienta que nos ha permitido comunicar entre la imaginación del equipo de diseño y la realidad del desarrollo.

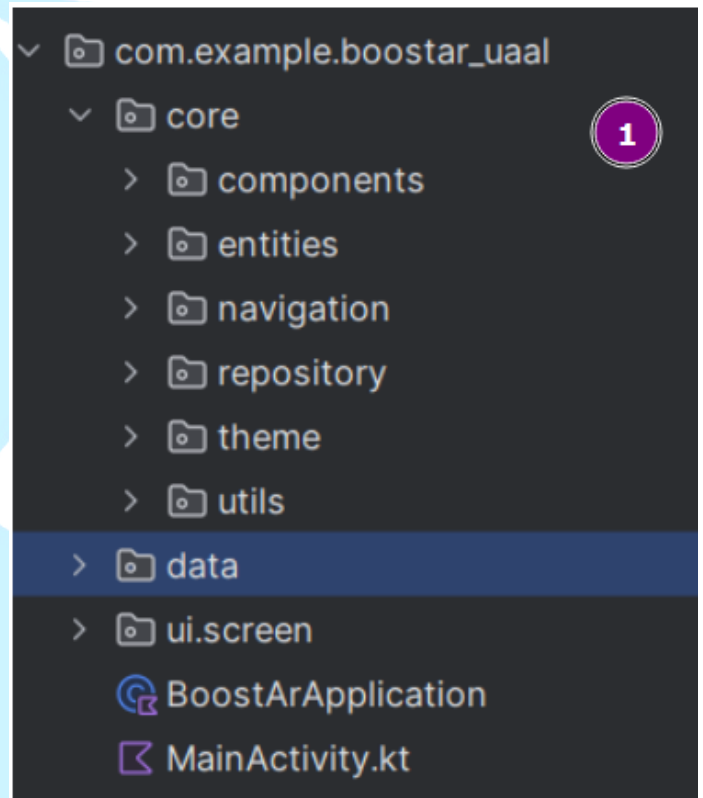
En una app donde pasan tantas cosas a la vez (cámara de Snapchat, login de Google, datos de Supabase), manejar qué se muestra en pantalla puede ser un caos. Compose se encarga de la recomposición automática; cuando los datos cambian, la interfaz se refresca sola. Compose redibuja solo la parte que lo necesita.



2.4.3- Separación de Componentes

Para garantizar que el proyecto fuera sostenible y escalable, aplicamos una **estricta separación de componentes**. El objetivo era evitar el "código acoplado" y asegurar que cada funcionalidad operara de manera independiente dentro del ecosistema de la app.

En la imagen 1 se observa una **arquitectura desacoplada**. Esto significa que si mañana decidimos cambiar Supabase por otro servicio, o si el equipo de diseño redefine un componente en Figma, el impacto en el código es mínimo y localizado, reduciendo drásticamente la posibilidad de introducir errores en cadena.



2.5- Diagramas de casos de uso

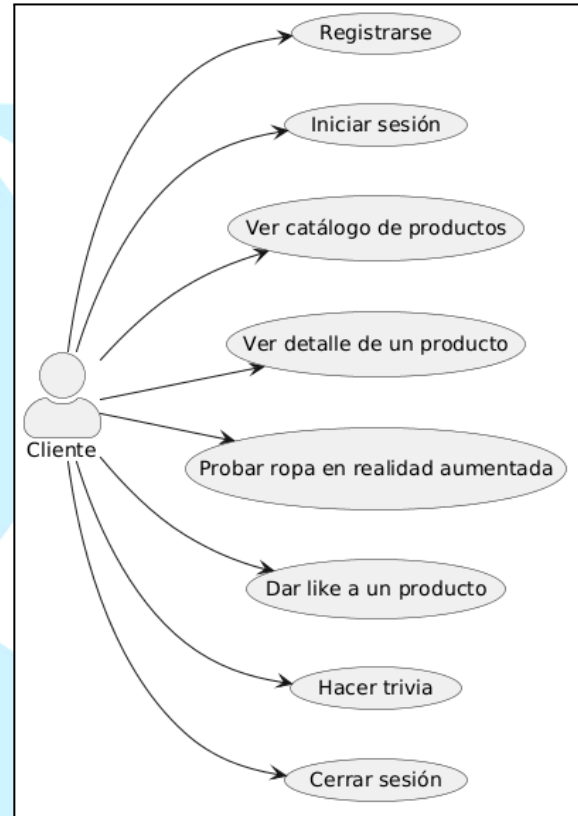
Este diagrama de casos de uso representa las funcionalidades que el Cliente puede realizar actualmente en la aplicación.

A futuro sería necesario implementar las siguientes funcionalidades:

- Compartir enlace del producto.
- Completar objetivos diarios, lecciones.
- Revisar áreas de conocimiento.
- Crear tus propios outfits.

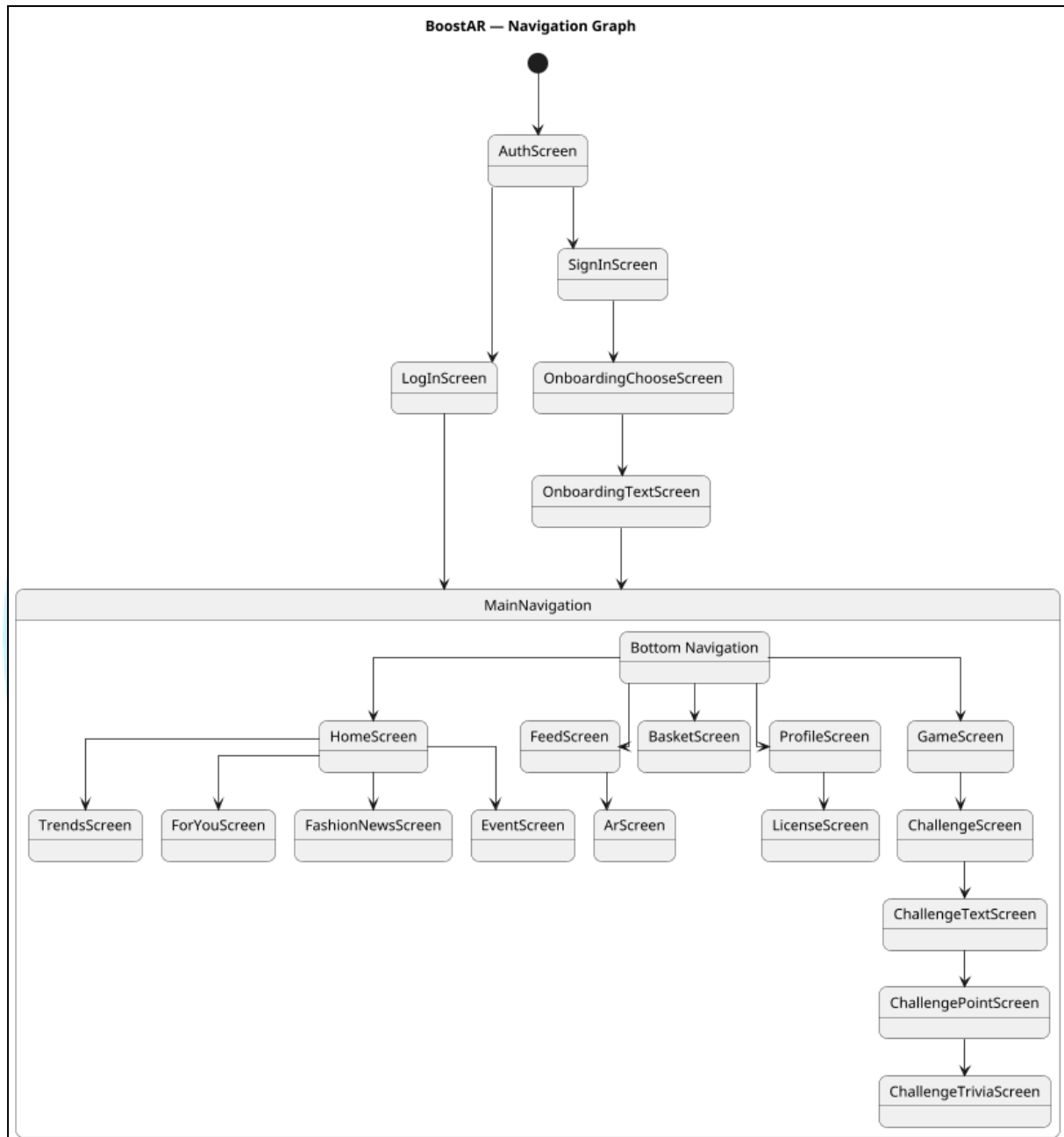
Cliente

- Añadir producto al carrito.
- Comprar productos.
- Visualizar historial de compra.
- Reservar un producto.





2.6- Flujo de la app





2.7- Bugs y posibles mejoras.

Estado de la paginación

Al acceder a la pantalla de AR desde la pantalla Feed, no se conserva el estado de la paginación.

En concreto, si el usuario entra en el Feed, hace scroll hacia abajo, prueba un producto en AR y después regresa al Feed, la paginación se reinicia, volviendo a mostrarse el producto inicial en lugar de mantener la posición previa.

Cargar productos previos.

En la pantalla Feed, no permite cargar los productos previos.

En concreto, si el usuario entra en el Feed, hace scroll hacia arriba, a la hora de cargar los productos devuelve una lista vacía.

Boton Provar AR

Sería interesante añadir un fallback en la pantalla AR, que detecte si no hay ningún modelo que mostrar te devuelva a la pantalla anterior.

Carga Pantalla Home

Actualmente, en dispositivos de bajo rendimiento, la pantalla Home tarda alrededor de 1 segundo en componerse, se debería buscar la causa del porqué para mejorar la UX.

Optimización del video

Un problema que se ha encontrado es que los videos consumen una gran cantidad de Gb de egress, se tendría que buscar una manera de reducirlo.

Hay dos soluciones:

- Alojarse los videos en un servicio aparte como CloudFlare.
- Buscar una manera de cachear los videos una única vez en la app y reutilizarlos localmente.



3- Dificultades

Comunicación entre equipo artístico y técnico

El choque entre los ideales de ambas partes, donde el equipo de diseño priorizaba un diseño bonito antes que la propia funcionalidad de la aplicación, mientras que el equipo de desarrollo priorizaba las funcionalidades sobre el diseño, prefiriendo añadir funcionalidades actualmente incompletas en la aplicación, como por ejemplo, la funcionalidad del carrito.

Trabajo en equipo

Uno de los principales obstáculos en el inicio fue el trabajo en equipo entre ambos desarrolladores, donde muchas veces había opiniones conflictivas sobre varias decisiones o situaciones que muchas veces se podrían haber afrontado desde un ángulo distinto.

Con el paso del tiempo, ambos desarrolladores hemos llegado a compaginarnos de manera adecuada, reduciendo los roces debido a los malentendidos.

Investigación AR

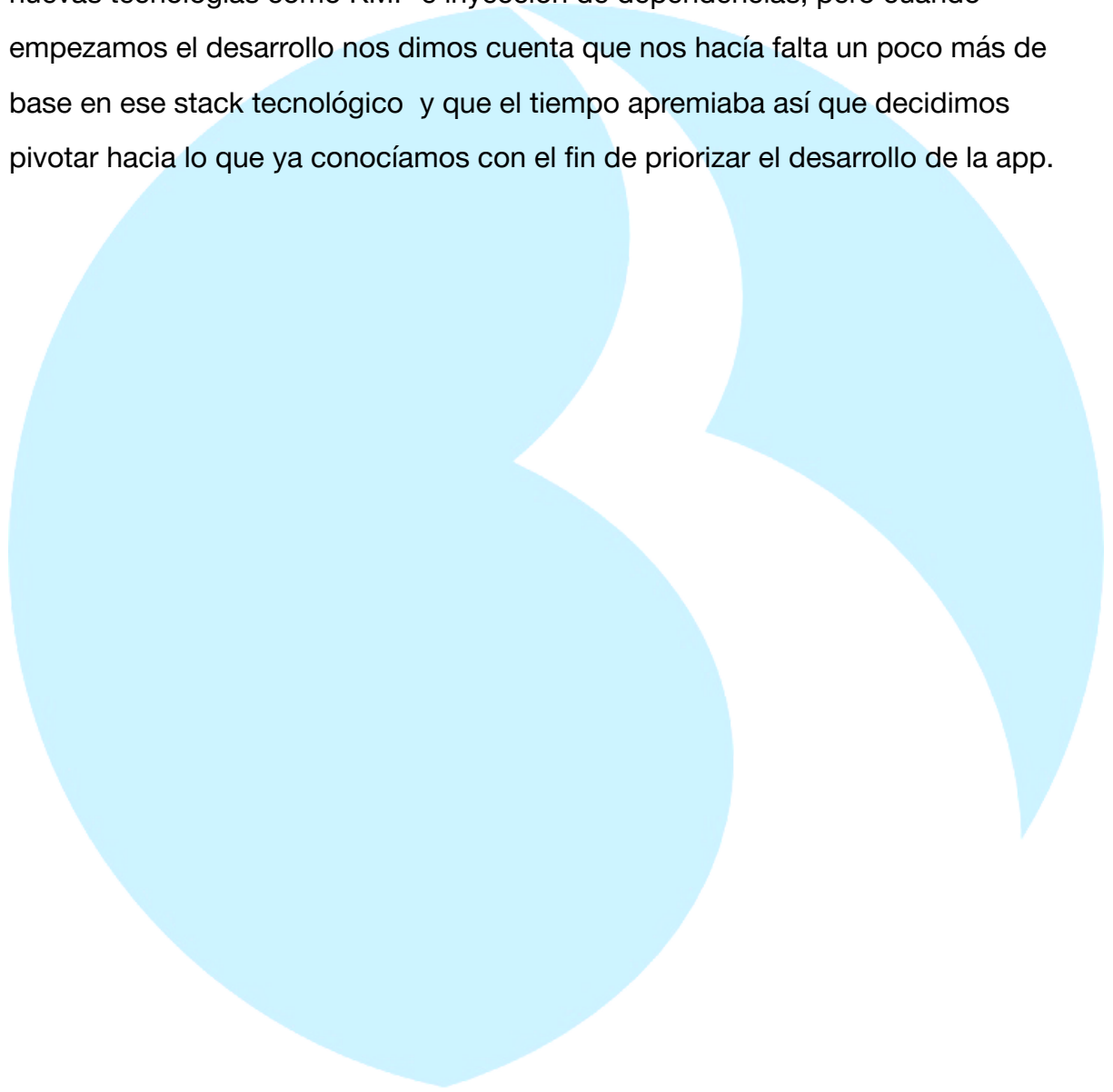
La parte de investigación sobre AR sin duda fue una sin no la que más tiempo consumió, muchas veces tuvimos que haber sido más pragmáticos y reconocer cuando una opción no daba más de sí.

Pasamos por múltiples opciones, entre ellas se encuentran el desarrollo de forma nativa mediante mediapipe y ARCore, que consumió alrededor de unos 3 meses de desarrollo, posteriormente continuamos con Unity y mediapipe retrasando el desarrollo por unos 2 meses y finalmente encontramos CameraKit, la opción más acertada, ya que proporcionaba una solución ya lista para usar, tardando en implementarlo no más de 1 semana.



Stack Tecnológico

Al inicio teníamos la sensación de tener mucho tiempo para levantar la app, y desarrollar todas funcionalidades , creyendo que seríamos capaces de aprender nuevas tecnologías como KMP e inyección de dependencias, pero cuando empezamos el desarrollo nos dimos cuenta que nos hacía falta un poco más de base en ese stack tecnológico y que el tiempo apremiaba así que decidimos pivotar hacia lo que ya conocíamos con el fin de priorizar el desarrollo de la app.





4- Conclusiones

Personalmente

A pesar de los múltiples obstáculos a lo largo del desarrollo, hemos aprendido a lidiar con ellos y a superarnos a nosotros mismos. Cada obstáculo nos ha ralentizado significativamente en el desarrollo, pero es innegable que nos ha aportado experiencia y conocimientos. Gracias a ellos hemos aprendido a hacer preguntas más concretas y lidiar con miembros del equipo, a expresarnos de una manera más diplomática y a ser más pragmáticos.

Proyecto

El Proyecto en su versión original que es la de desarrollar el AR nativamente es bastante ambicioso y extenso de desarrollar, nosotros hemos empezado el recorrido y hemos aportado una primera versión muy mejorable pero funcional, esta sería la prueba que indica que es posible su desarrollo teniendo en cuenta más recursos, tiempo y conocimientos.